

TRACE-LAB – Sistema de Rastreabilidade e Gestão de Processos Laboratoriais

João Kasprowicz

2026

1. INTRODUÇÃO

A informatização dos processos laboratoriais é um elemento essencial para ampliar a segurança, a rastreabilidade e a padronização das operações. Em atividades que envolvem coleta, transporte e recebimento de amostras biológicas, o registro estruturado de eventos reduz falhas operacionais, fortalece o histórico das movimentações e melhora a qualidade da informação disponível para tomada de decisão.

No contexto desse projeto, a rastreabilidade foi observada principalmente na etapa logística entre pontos de coleta e o recebimento final no laboratório. O TRACE-LAB surge como uma proposta aplicada para registrar esse percurso, preservando dados como identificação da rota, tipo de veículo, turno, temperatura e condição das amostras.

2. BREVE DESCRIÇÃO DO PROJETO

O TRACE-LAB é um sistema full stack composto por backend em Django REST Framework e aplicativo móvel em Flutter. Seu propósito, no estado atual do código, é apoiar a gestão operacional do transporte de amostras, com foco em rastreabilidade, segregação de papéis e registro cronológico dos eventos associados à rota.

- Problema tratado: falta de controle estruturado sobre transporte, coletas intermediárias e recebimento final.
- Público-alvo: motoristas, recebedores e usuários com perfil administrativo.
- Contexto de aplicação: ambientes laboratoriais ou serviços que dependem de transporte controlado de amostras.
- Benefícios esperados: histórico operacional, maior visibilidade do fluxo e apoio ao controle de temperatura e integridade.
- Relação com processos laboratoriais: a solução atua na etapa logística, antes do processamento analítico propriamente dito.

3. OBJETIVO DO SISTEMA

A análise do repositório indica que o objetivo central do sistema é organizar e rastrear a operação de transporte de amostras por meio de registros estruturados e separados por perfil de usuário.

- Iniciar e registrar rotas de transporte.
- Identificar veículo, turno e bolsa vinculados à rota.
- Registrar coletas com local, temperatura e observações.
- Manter o histórico cronológico das coletas realizadas.
- Permitir o encerramento formal da rota pelo motorista.
- Disponibilizar rotas finalizadas para o perfil de recebimento.
- Registrar temperatura de recebimento e integridade das amostras.
- Preservar a rastreabilidade e a segregação entre fluxos operacionais.

Não foram identificadas implementações completas para cadastro de pacientes, exames, resultados ou emissão de laudos. Esses pontos podem ser tratados como possibilidades de evolução, mas não como funcionalidades concluídas na versão analisada.

4. TECNOLOGIAS UTILIZADAS

Tecnologia	Finalidade	Observação
Django	Estrutura do backend	Projeto configurado em backend/config/settings.py
Django REST Framework	Exposição da API REST	Uso de APIView, serializers e respostas JSON
Simple JWT	Autenticação	Login e refresh de token JWT
Flutter	Aplicativo cliente	Interface mobile para motorista e recebedor
HTTP package	Consumo da API	Integração do app Flutter com a API REST
SQLite	Persistência	Banco configurado para desenvolvimento local
CORS Headers	Integração local	Liberação de origem localhost para o app
GitHub	Versionamento e portfólio	Repositório remoto identificado no origin

REST API	Comunicação entre camadas	Endpoints sob o prefixo /api
----------	---------------------------	------------------------------

5. ARQUITETURA DA SOLUÇÃO

A arquitetura adotada é do tipo cliente-servidor, com aplicação móvel consumindo uma API REST protegida por autenticação JWT. No backend, o projeto está organizado em apps Django, enquanto o frontend utiliza arquitetura por funcionalidades, separando serviços, modelos, DTOs, telas e widgets.

- Backend: Django, Django REST Framework, autenticação Simple JWT e controle de permissões por papel.
- Frontend: Flutter com consumo HTTP da API e separação modular por feature.
- Banco de dados: SQLite configurado para o ambiente atual do projeto.
- API: endpoints REST sob o prefixo /api para autenticação, rotas, coletas e recebimento.
- Módulos principais: accounts e routes_app no backend; auth, start_route, active_route, collection, receiver e receiving no frontend.

Diagrama textual da solução:

- Usuário
- Interface do sistema em Flutter
- API REST em Django REST Framework
- Camada de regras de acesso e validação
- Banco de dados SQLite

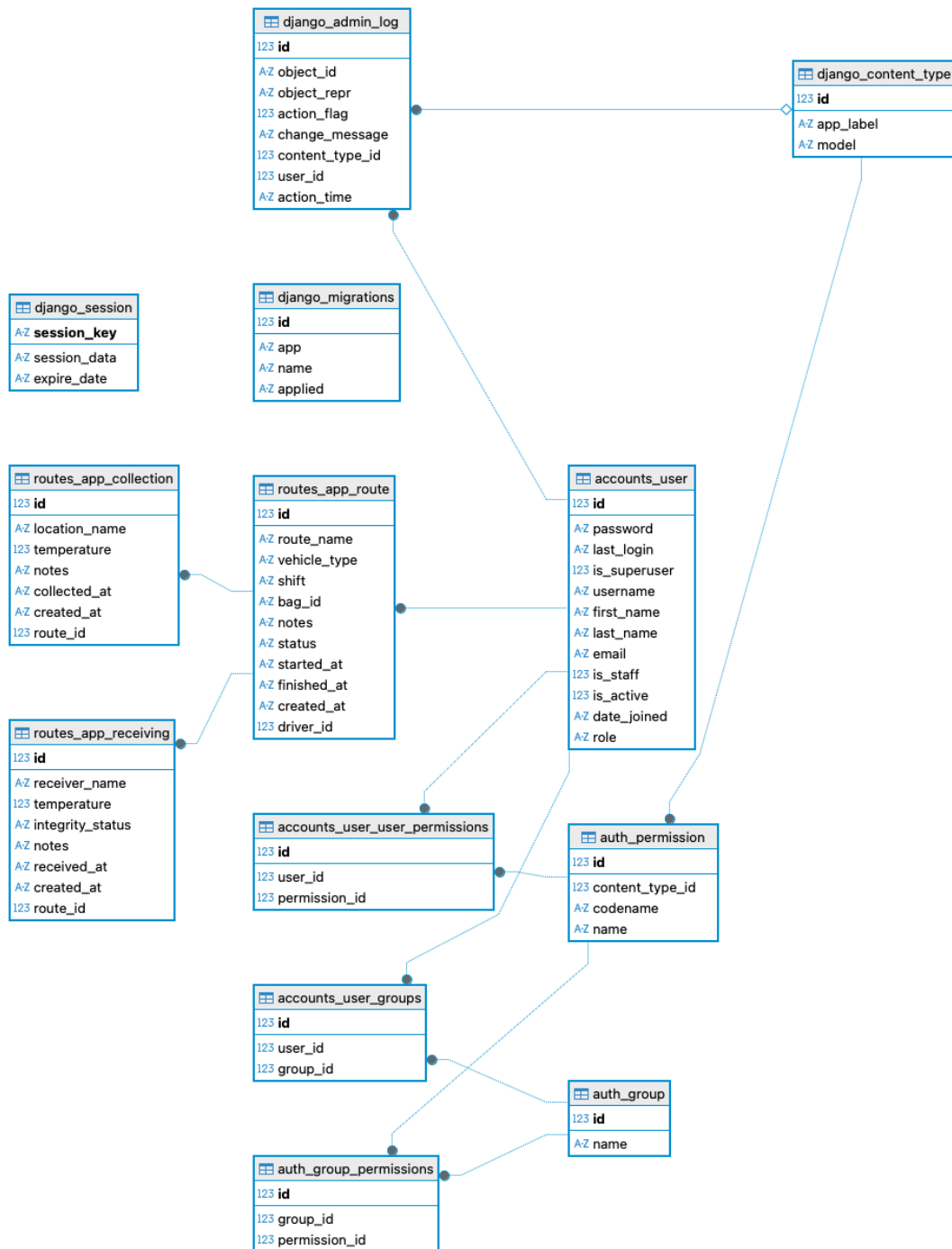


Figura 1 – Modelo entidade-relacionamento do TRACE-LAB

6. PRINCIPAIS FUNCIONALIDADES IDENTIFICADAS

Módulo	Funcionalidades identificadas
Autenticação	Login, emissão de JWT, refresh de token e encaminhamento por perfil.

Motorista	Início de rota, registro de coletas, visualização de linha do tempo e finalização da rota.
Recebedor	Listagem de rotas pendentes, recebimento, temperatura de chegada e integridade das amostras.
Administração	Dashboard administrativo inicial com acesso por papel.

O modelo de dados observado no MER reforça o escopo atual do sistema. Há uma relação de um usuário para várias rotas, de uma rota para várias coletas e de uma rota para um único recebimento, o que representa de forma consistente o ciclo de transporte identificado no código.

Como achados complementares da análise, observou-se que a suíte de testes ainda está pouco evoluída e que o painel administrativo permanece inicial. Ainda assim, a base arquitetural é adequada para uso acadêmico, demonstração técnica e futuras expansões do produto.